

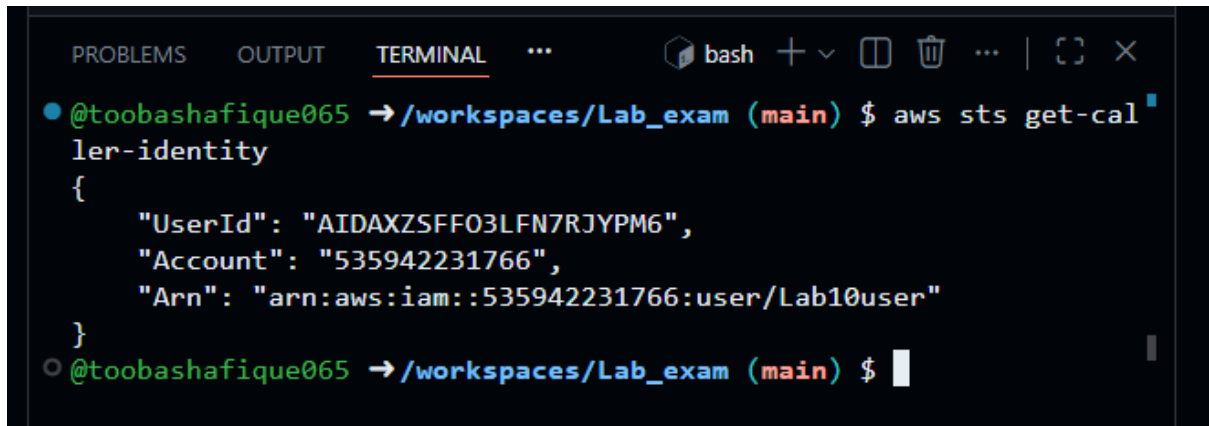
Tooba Shafique

Cloud Computing

2023-BSE-065

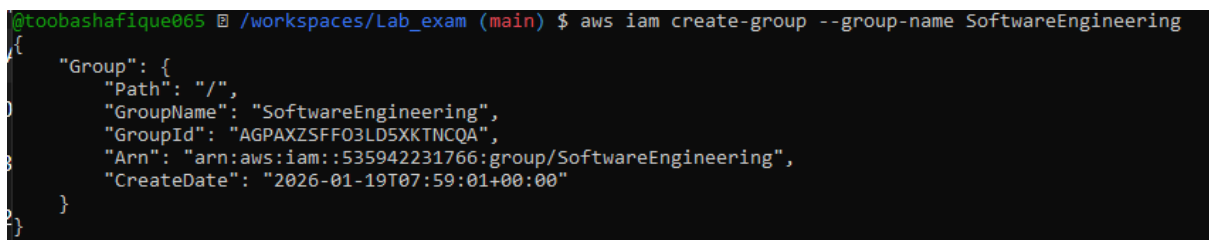
BSE-VB

Qno1



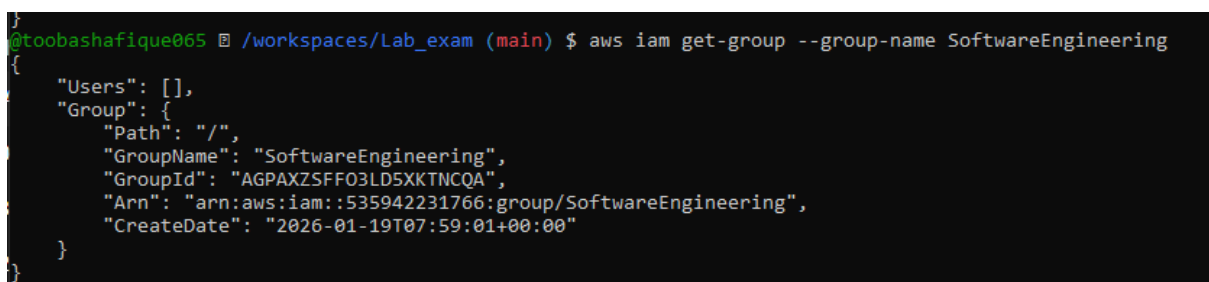
```
PROBLEMS OUTPUT TERMINAL ... bash + v [ ] [ ] ... | [ ] [ ] x
@toobashafique065 → /workspaces/Lab_exam (main) $ aws sts get-caller-identity
{
  "UserId": "AIDAXZSFF03LFN7RJYPM6",
  "Account": "535942231766",
  "Arn": "arn:aws:iam::535942231766:user/Lab10user"
}
@toobashafique065 → /workspaces/Lab_exam (main) $
```

q1_create_group.png



```
@toobashafique065 → /workspaces/Lab_exam (main) $ aws iam create-group --group-name SoftwareEngineering
{
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXZSFF03LD5XKTNCQA",
    "Arn": "arn:aws:iam::535942231766:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:59:01+00:00"
  }
}
```

q1_group_details.png



```
@toobashafique065 → /workspaces/Lab_exam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXZSFF03LD5XKTNCQA",
    "Arn": "arn:aws:iam::535942231766:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:59:01+00:00"
  }
}
```

q1_create_user.png

```

@toobashafique065 [ ] /workspaces/Lab_exam (main) $ aws iam create-user --user-name Tooba
{
  "User": {
    "Path": "/",
    "UserName": "Tooba",
    "UserId": "AIDAXZSFF03LE2FBQPUS7",
    "Arn": "arn:aws:iam::535942231766:user/Tooba",
    "CreateDate": "2026-01-19T07:59:54+00:00"
  }
}
@toobashafique065 [ ] /workspaces/Lab_exam (main) $

```

q1_user_details.png

```

@toobashafique065 [ ] /workspaces/Lab_exam (main) $ aws iam get-user --user-name Tooba
{
  "User": {
    "Path": "/",
    "UserName": "Tooba",
    "UserId": "AIDAXZSFF03LE2FBQPUS7",
    "Arn": "arn:aws:iam::535942231766:user/Tooba",
    "CreateDate": "2026-01-19T07:59:54+00:00"
  }
}
@toobashafique065 [ ] /workspaces/Lab_exam (main) $

```

q1_add_user_to_group.png

```

@toobashafique065 [ ] /workspaces/Lab_exam (main) $ aws iam add-user-to-group \
> --user-name Tooba \
> --group-name SoftwareEngineering
@toobashafique065 [ ] /workspaces/Lab_exam (main) $

```

q1_group_membership.png

```

@toobashafique065 [ ] /workspaces/Lab_exam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Tooba",
      "UserId": "AIDAXZSFF03LE2FBQPUS7",
      "Arn": "arn:aws:iam::535942231766:user/Tooba",
      "CreateDate": "2026-01-19T07:59:54+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPAXZSFF03LD5XKTCQA",
    "Arn": "arn:aws:iam::535942231766:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:59:01+00:00"
  }
}
@toobashafique065 [ ] /workspaces/Lab_exam (main) $

```

q1_find_admin_policy.png

```

@toobashafique065 [ /workspaces/Lab_exam (main) $ aws iam list-policies --scope AWS \
> --query "Policies[?PolicyName=='AdministratorAccess']"
[
  {
    "PolicyName": "AdministratorAccess",
    "PolicyId": "ANPAIWMBCKSKIEE64ZLYK",
    "Arn": "arn:aws:iam::aws:policy/AdministratorAccess",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 3,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2015-02-06T18:39:46+00:00",
    "UpdateDate": "2015-02-06T18:39:46+00:00"
  }
]

```

q1_attach_admin_policy.png

```

@toobashafique065 [ /workspaces/Lab_exam (main) $ aws iam attach-group-policy \
> --group-name SoftwareEngineering \
> --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

```

q1_list_group_policies.png

```

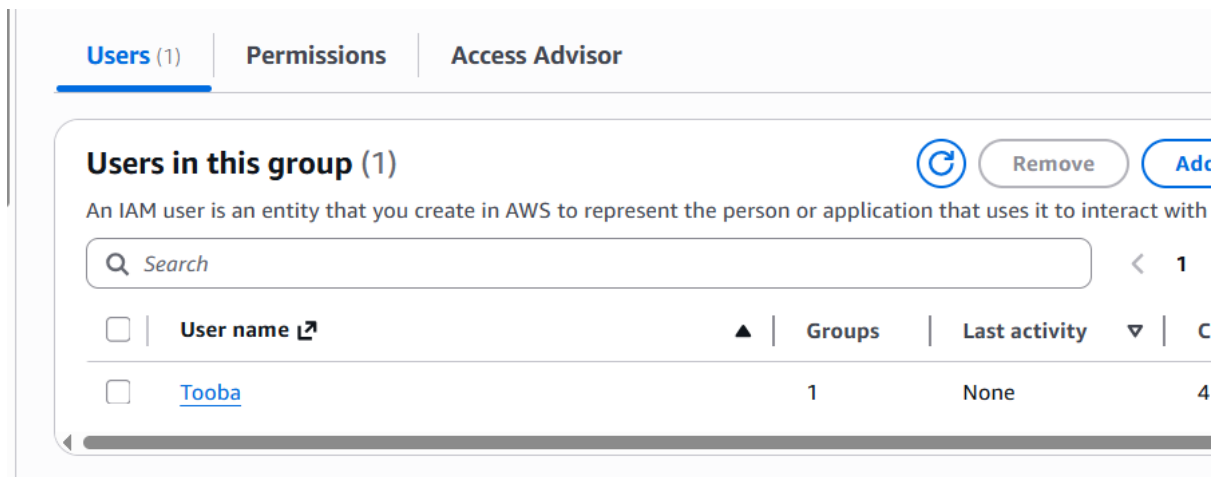
@toobashafique065 [ /workspaces/Lab_exam (main) $ aws iam list-attached-group-policies \
> --group-name SoftwareEngineering
{
  "AttachedPolicies": [
    {
      "PolicyName": "AdministratorAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    }
  ]
}

```

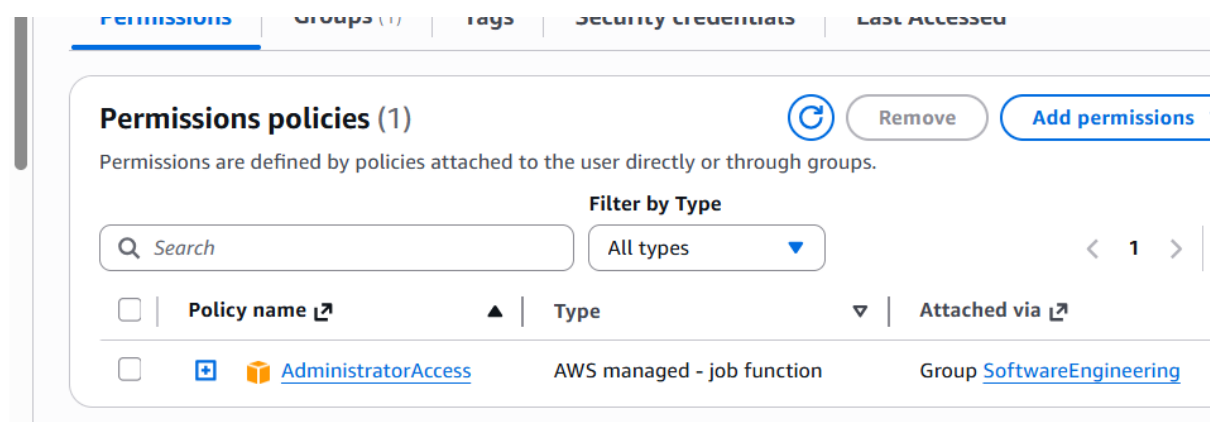
q1_console_group.png

	Search			
☐	Group name	▲	Users	▼ Permissions ▼
☐	SoftwareEngineering		1	✔ Defined

q1_console_user_in_group.png

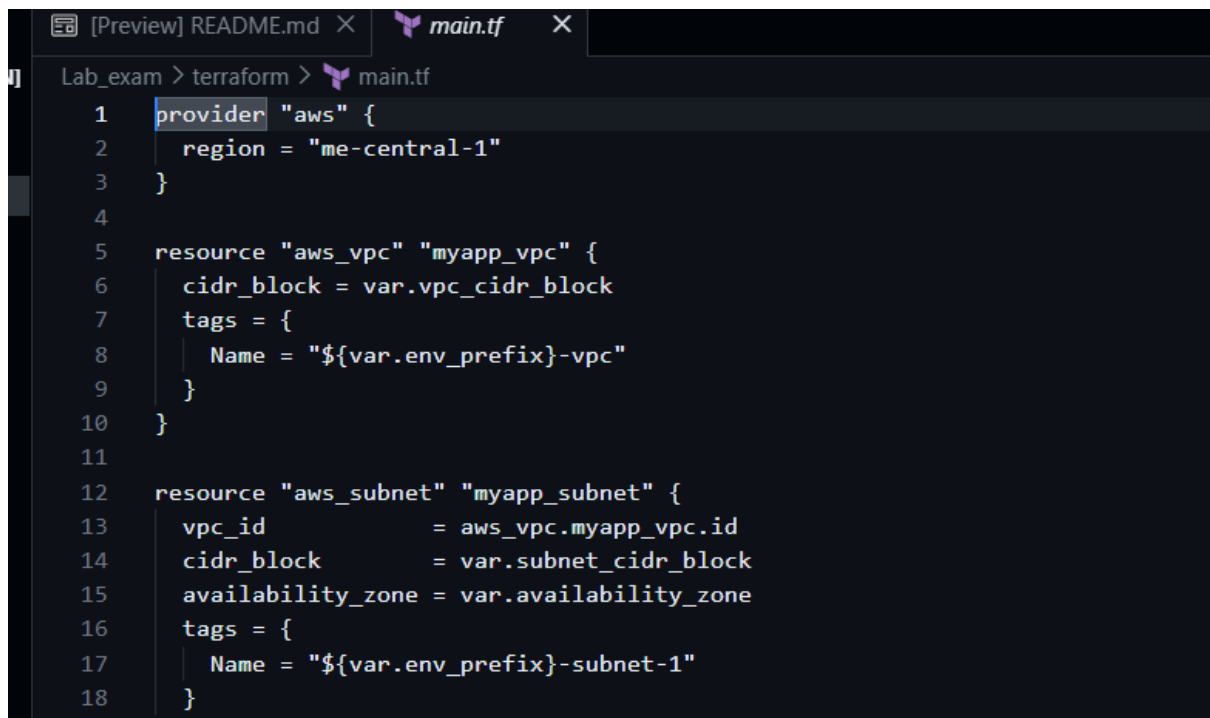


q1_console_group_policy.png



QNO 2:

- q2_provider.png



The screenshot shows a code editor with two tabs: "[Preview] README.md" and "main.tf". The "main.tf" tab is active, displaying Terraform configuration code. The code defines an AWS provider, an AWS VPC resource named "myapp_vpc", and an AWS Subnet resource named "myapp_subnet". The VPC resource is configured with a region of "me-central-1", a CIDR block from a variable, and a tag. The Subnet resource is configured with the VPC ID, a CIDR block from a variable, an availability zone from a variable, and a tag. The code is as follows:

```
1 provider "aws" {
2   region = "me-central-1"
3 }
4
5 resource "aws_vpc" "myapp_vpc" {
6   cidr_block = var.vpc_cidr_block
7   tags = {
8     Name = "${var.env_prefix}-vpc"
9   }
10 }
11
12 resource "aws_subnet" "myapp_subnet" {
13   vpc_id          = aws_vpc.myapp_vpc.id
14   cidr_block       = var.subnet_cidr_block
15   availability_zone = var.availability_zone
16   tags = {
17     Name = "${var.env_prefix}-subnet-1"
18   }
19 }
```

- q2_igw_route_table.png



The screenshot shows a code editor with a single tab containing Terraform configuration code. The code defines an AWS Internet Gateway resource named "igw" and an AWS Default Route Table resource named "rt". The Internet Gateway resource is configured with a tag. The Default Route Table resource is configured with a default route table ID from the VPC resource, a route to the Internet Gateway, and a tag. The code is as follows:

```
resource "aws_internet_gateway" "igw" {
}

resource "aws_default_route_table" "rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
```

- q2_vpc_subnet.png

```

  ✓ resource "aws_vpc" "myapp_vpc" {
    cidr_block = var.vpc_cidr_block
  ✓   tags = {
      Name = "${var.env_prefix}-vpc"
    }
  }

  ✓ resource "aws_subnet" "myapp_subnet" {
    vpc_id          = aws_vpc.myapp_vpc.id
    cidr_block      = var.subnet_cidr_block
    availability_zone = var.availability_zone
  ✓   tags = {
      Name = "${var.env_prefix}-subnet-1"
    }
  }

```

- q2_http_and_locals.png

```

ab_exam > terraform > ip.tf
1  data "http" "myip" {
2    url = "https://icanhazip.com"
3  }
4
5  locals {
6    my_ip = "${chomp(data.http.myip.response_body)}/32"
7  }
8

```

- q2_default_sg.png

[Preview] README.md ec2.tf security.tf U X

Lab_exam > terraform > security.tf

```
1 resource "aws_default_security_group" "default_sg" {
2     vpc_id = aws_vpc.myapp_vpc.id
3
4     ingress {
5         from_port    = 22
6         to_port      = 22
7         protocol      = "tcp"
8         cidr_blocks   = [local.my_ip]
9     }
10
11    ingress {
12        from_port    = 80
13        to_port      = 80
14        protocol      = "tcp"
15        cidr_blocks   = ["0.0.0.0/0"]
16    }
17
18    ingress {
```

```
    ingress {
        from_port    = 443
        to_port      = 443
        protocol      = "tcp"
        cidr_blocks   = ["0.0.0.0/0"]
    }

    egress {
        from_port    = 0
        to_port      = 0
        protocol      = "-1"
        cidr_blocks   = ["0.0.0.0/0"]
    }

    tags = {
        Name = "${var.env_prefix}-default-sg"
    }
}
```

```
tags = {
  Name = "${var.env_prefix}-default-sg"
}
```

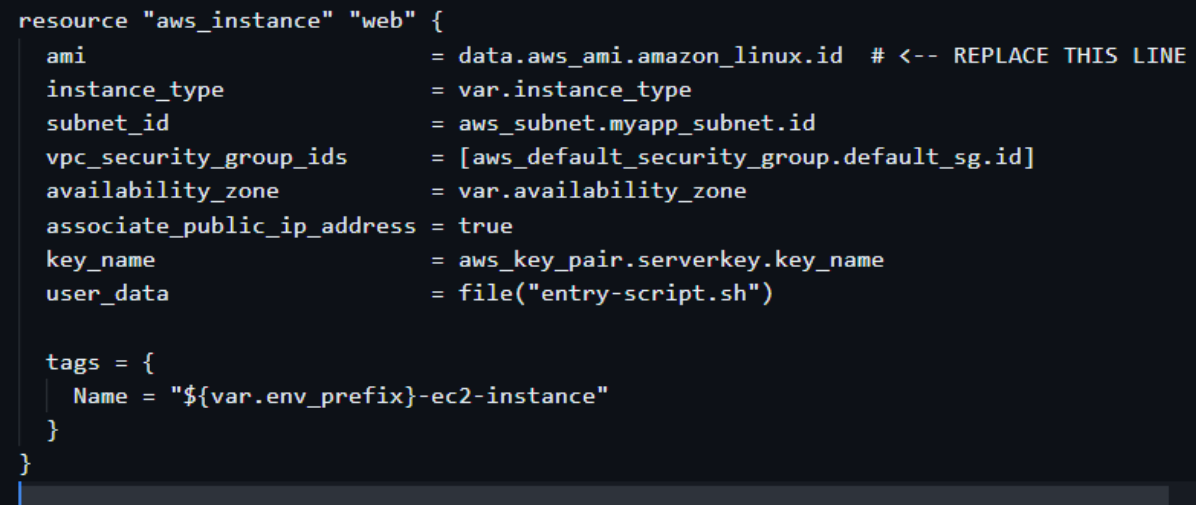
- q2_keypair.png



The screenshot shows a code editor with three tabs: "[Preview] README.md", "ec2.tf", and "keypair.tf". The "keypair.tf" tab is active, showing the following Terraform configuration:

```
1 resource "aws_key_pair" "serverkey" {
2     key_name     = "serverkey"
3     public_key  = file("~/ssh/id_ed25519.pub")
4 }
5
```

- q2_ec2_resource.png



The screenshot shows a code editor with a single tab containing the following Terraform configuration for an AWS EC2 instance:

```
resource "aws_instance" "web" {
  ami                    = data.aws_ami.amazon_linux.id # <-- REPLACE THIS LINE
  instance_type         = var.instance_type
  subnet_id             = aws_subnet.myapp_subnet.id
  vpc_security_group_ids = [aws_default_security_group.default_sg.id]
  availability_zone      = var.availability_zone
  associate_public_ip_address = true
  key_name              = aws_key_pair.serverkey.key_name
  user_data             = file("entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

- q2_entry_script.png

```

@toobashafique005 /workspaces/Lab_exam (main) $ cd Lab_exam/terraform
@toobashafique065 /workspaces/Lab_exam/Lab_exam/terraform (main) $ cat entry-script.sh
#!/bin/bash
dnf update -y
dnf install -y nginx openssl

mkdir -p /etc/nginx/ssl

openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/nginx.key \
  -out /etc/nginx/ssl/nginx.crt \
  -subj "/CN=localhost"

cat > /etc/nginx/conf.d/default.conf <<EOF
server {
    listen 80;
    return 301 https://\$host\$request_uri;
}
server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/nginx.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
EOF

echo "This is Tooba's Terraform environment" > /usr/share/nginx/html/index.html

systemctl enable nginx
systemctl restart nginx

```

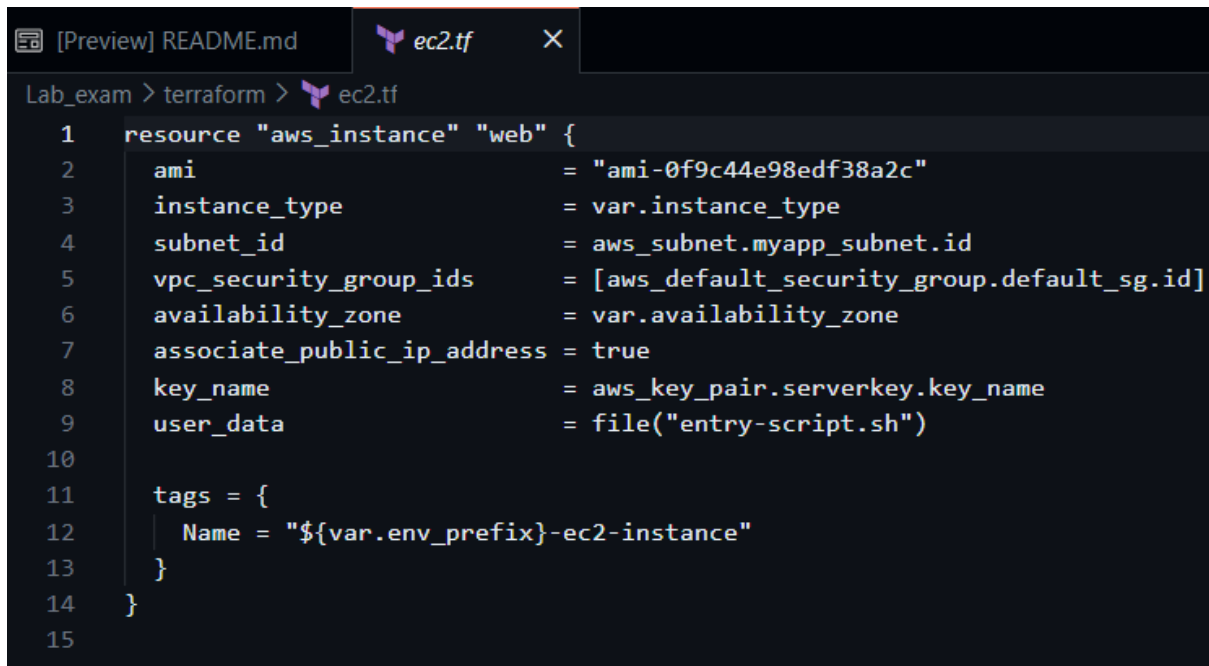
- q2_variables.png

```

Lab_exam > terraform > variables.tf
1  variable "vpc_cidr_block" {}
2  variable "subnet_cidr_block" {}
3  variable "availability_zone" {}
4  variable "env_prefix" {}
5  variable "instance_type" {}
6

```

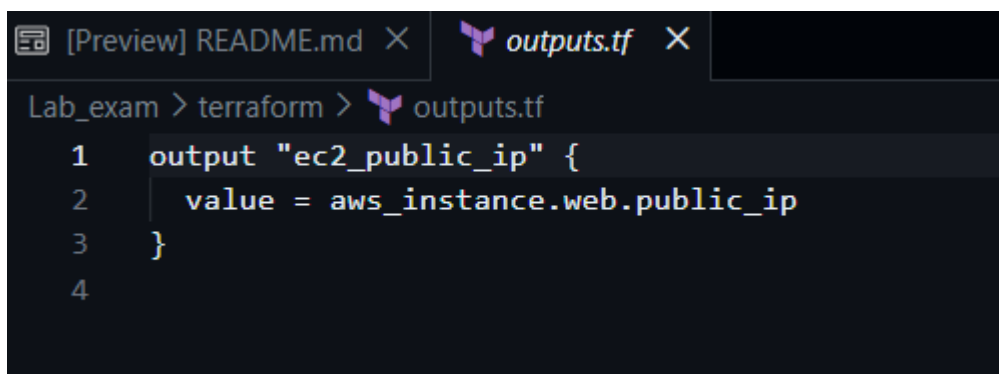
- q2_ec2_resource.png



The screenshot shows a code editor with a tab labeled 'ec2.tf'. The file content is a Terraform resource definition for an AWS EC2 instance named 'web'. The configuration includes attributes for ami, instance_type, subnet_id, vpc_security_group_ids, availability_zone, associate_public_ip_address, key_name, user_data, and tags. The tags block specifies a Name tag using a variable.

```
1 resource "aws_instance" "web" {
2     ami                = "ami-0f9c44e98edf38a2c"
3     instance_type      = var.instance_type
4     subnet_id          = aws_subnet.myapp_subnet.id
5     vpc_security_group_ids = [aws_default_security_group.default_sg.id]
6     availability_zone   = var.availability_zone
7     associate_public_ip_address = true
8     key_name            = aws_key_pair.serverkey.key_name
9     user_data           = file("entry-script.sh")
10
11     tags = {
12         Name = "${var.env_prefix}-ec2-instance"
13     }
14 }
15
```

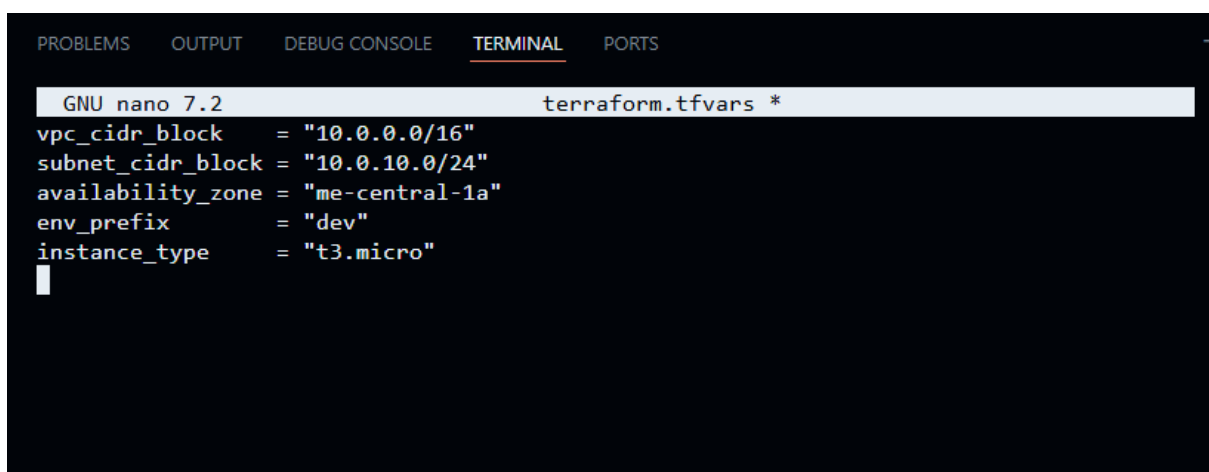
- q2_output_block.png



The screenshot shows a code editor with a tab labeled 'outputs.tf'. The file content is a Terraform output definition for 'ec2_public_ip', which is set to the public_ip attribute of the 'aws_instance.web' resource.

```
1 output "ec2_public_ip" {
2     value = aws_instance.web.public_ip
3 }
4
```

- q2_tfvars_or_vars.png



The screenshot shows a terminal window with the title 'terraform.tfvars *'. The content lists several variables and their values, including vpc_cidr_block, subnet_cidr_block, availability_zone, env_prefix, and instance_type.

```
GNU nano 7.2 terraform.tfvars *
vpc_cidr_block    = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone  = "me-central-1a"
env_prefix        = "dev"
instance_type     = "t3.micro"
|
```

- q2_terraform_init.png

```
@toobashafique065 →/workspaces/Lab_exam/Lab_exam/terraform (main) $ terraform init
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

- q2_terraform_plan.png

```
@toobashafique065 →/workspaces/Lab_exam/Lab_exam/terraform (main) $ terraform plan
data.http.myip: Reading...
data.http.myip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# aws_default_route_table.rt will be created
+ resource "aws_default_route_table" "rt" {
  + arn                = (known after apply)
  + default_route_table_id = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + region              = "me-central-1"
  + route               = [
    + {
      + cidr_block          = "0.0.0.0/0"
      + gateway_id          = (known after apply)
      # (10 unchanged attributes hidden)
    }
  ]
}
```

- q2_terraform_apply.png

```
@toobashafique065 → /workspaces/Lab_exam/Lab_exam/terraform (main) $ terraform apply -auto-approve

+ network_interface (known after apply)

+ primary_network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 14s [id=i-0a9ab5a9455662e35]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

ec2_public_ip = "51.112.186.20"
@toobashafique065 → /workspaces/Lab_exam/Lab_exam/terraform (main) $
```

- q2_terraform_output.png

```
Outputs:

ec2_public_ip = "51.112.186.20"
@toobashafique065 → /workspaces/Lab_exam/Lab_exam/terraform (main) $
```

- q2_console_vpc.png

Your VPCs (3) [Info](#) Last updated less than a minute ago [Actions](#) [Create VPC](#)

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...
☐	-	vpc-0b8fa1602193cc7f7	✓ Available	-
☐	labproj-vpc	vpc-0880d3593b16518a7	✓ Available	-
☐	dev-vpc	vpc-04f7035061f283ce6	✓ Available	-

- q2_console_subnet.png

Find subnets by attribute or tag							
☐	Name	Subnet ID	State	VPC	Bloc...	IPv4 CIDR	
☐	-	subnet-032c91...	✓ Available.	vpc-0880d...	⊖ Off	10.0.1.0/24	
☐	-	subnet-0d3e13...	✓ Available.	vpc-0b8fa...	⊖ Off	172.31.16.0	
☐	-	subnet-0a589a...	✓ Available.	vpc-0b8fa...	⊖ Off	172.31.0.0/24	
☐	dev-subnet-1	subnet-0e992c...	✓ Available.	vpc-04f70...	⊖ Off	10.0.10.0/24	
☐	-	subnet-0464b8...	✓ Available.	vpc-0b8fa...	⊖ Off	172.31.32.0	

- q2_console_route_table.png

Find route tables by attribute or tag			
Name	Route table ID	Explicit subnet associ...	
-	rtb-0d8a4ae686234a85c	-	
-	rtb-08e8e4a442ceb01c0	-	
dev-rt	rtb-0db235a90bab8dd02	-	
-	rtb-0b6a594de53fae36e	subnet-032c915dfa0610...	

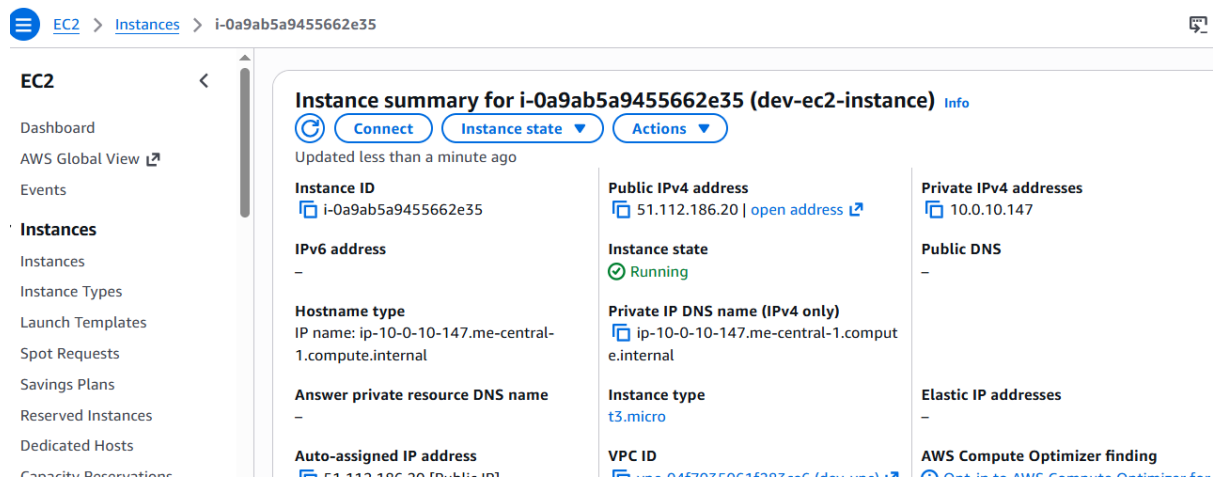
- q2_console_igw.png

Find internet gateways by attribute or tag				
Name	Internet gateway ID	State	VPC ID	
-	igw-03d5e42e9d6cc7daf	✓ Attached	vpc-08i	
-	igw-0df9068e01d80ed62	✓ Attached	vpc-0b	
dev-igw	igw-0ee4a0d37dbd90021	✓ Attached	vpc-04f	

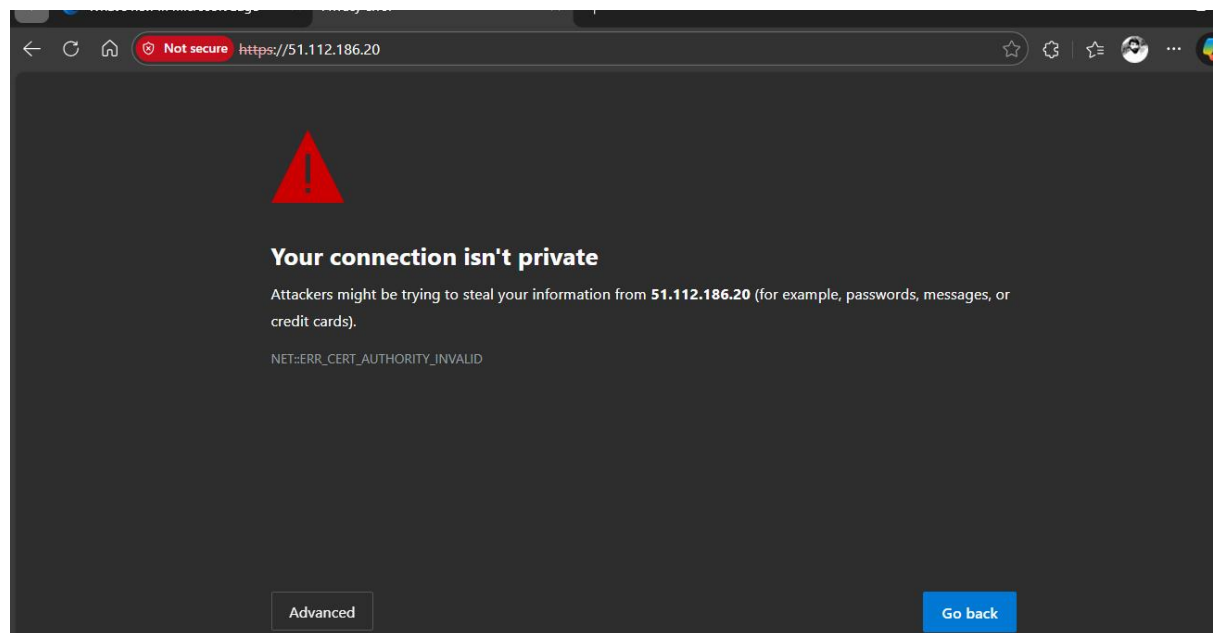
- q2_console_sg.png

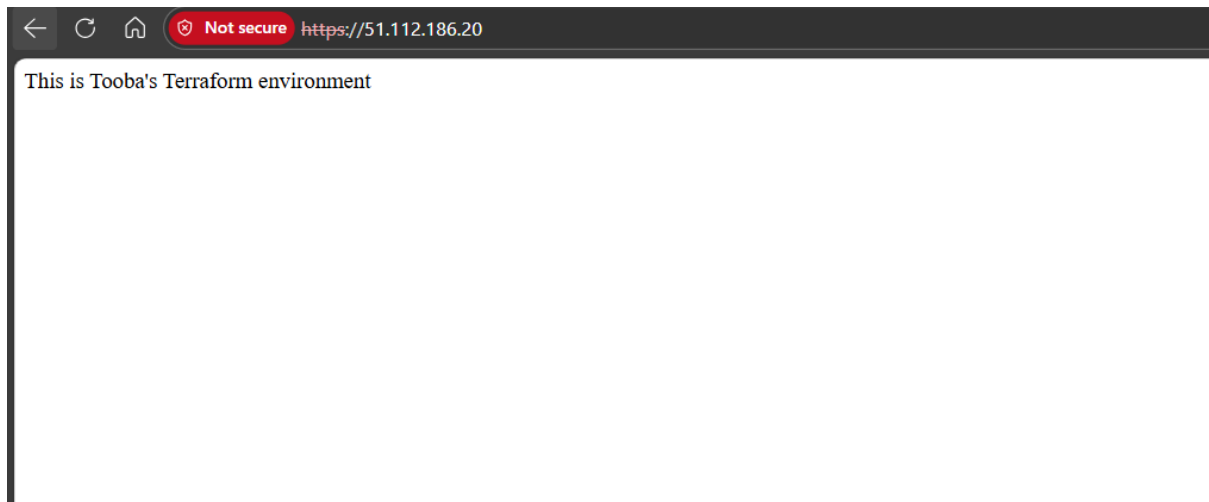
Find security groups by attribute or tag				
Name	Security group ID	Security group name	VPC ID	
-	sg-04b445d59ec742378	labproj-sg	vpc-08	
-	sg-015d6af85e8fba4b	default	vpc-0b	
-	sg-0b39aa7a34f7f76c5	default	vpc-08	
dev-default-sg	sg-086550a40a2daf9f1	default	vpc-02	

- q2_console_ec2.png



- q2_https_browser.png





QNO3:

- q3_hosts.png

A screenshot of a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal shows the command 'cat /etc/hosts' being executed, resulting in the output '51.112.186.20 ec2'. Below this, the command 'cat /etc/ansible/hosts' is executed, showing the contents of the hosts file: '[ec2:vars]', 'ansible_user=ec2-user', 'ansible_ssh_private_key_file=~/.ssh/id_ed25519', and 'ansible_ssh_common_args=\'-o StrictHostKeyChecking=no\''.

```
GNU nano 7.2 hosts
[ec2]
51.112.186.20

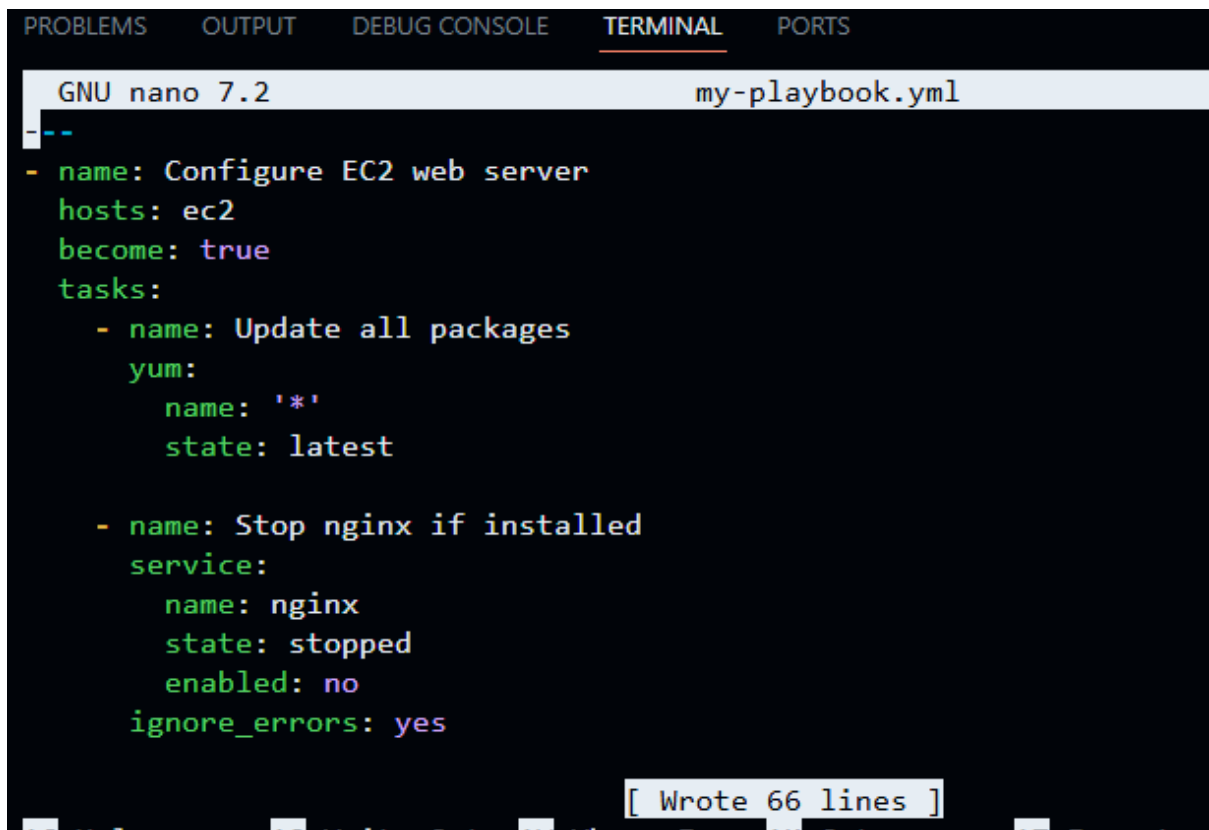
[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

- q3_ansible_cfg.png

A screenshot of a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal shows the command 'cat /etc/ansible/ansible.cfg' being executed, resulting in the output of the ansible.cfg file: '[defaults]', 'host_key_checking = False', 'interpreter_python = /usr/bin/python3', and 'inventory = ./hosts'.

```
GNU nano 7.2 ansible.cfg *
[defaults]
host_key_checking = False
interpreter_python = /usr/bin/python3
inventory = ./hosts
```

- q3_playbook.png



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GNU nano 7.2 my-playbook.yml
--
- name: Configure EC2 web server
  hosts: ec2
  become: true
  tasks:
    - name: Update all packages
      yum:
        name: '*'
        state: latest

    - name: Stop nginx if installed
      service:
        name: nginx
        state: stopped
        enabled: no
        ignore_errors: yes

[ Wrote 66 lines ]
```

My ip is changed here form before because i ran terraform apply again.

- q3_http_browser.png

